

NAMOSIM: A Robot Motion Planner for Navigation Among Movable Obstacles

David Brown^{1 4}, Jacques Saraydaryan^{1 3 4}, Benoit Renault^{2 4}, and Olivier Simonin^{1 2 4}

1 Inria, CHROMA Team 2 INSA Lyon 3 CPE Lyon 4 CITI Laboratory

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- Review [↗](#)
- Repository [↗](#)
- Archive [↗](#)

Editor: [Open Journals](#) [↗](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

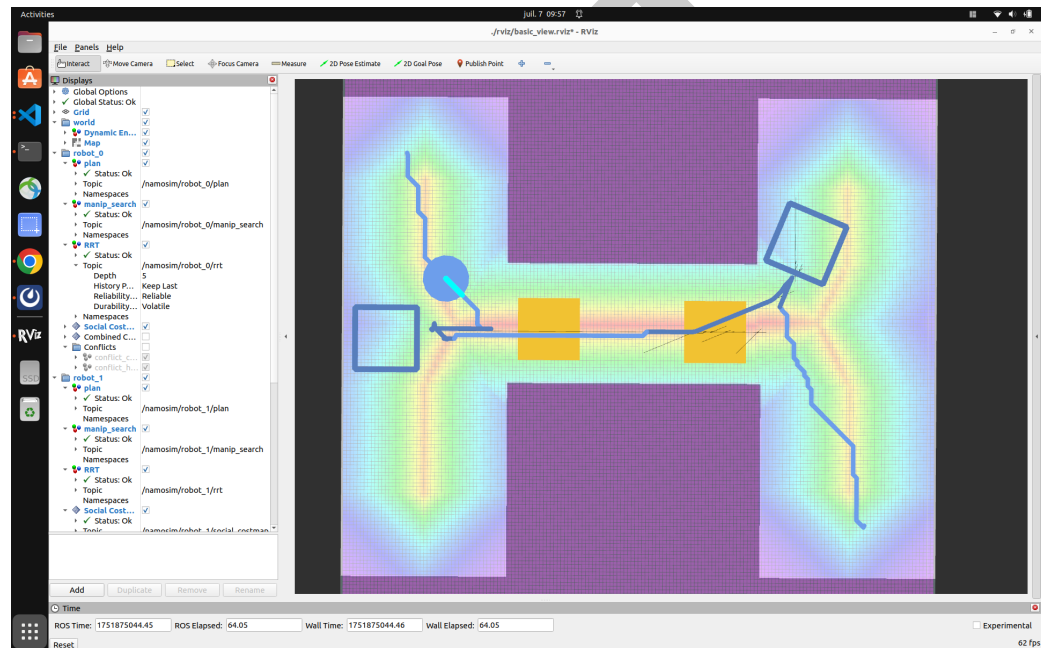


Figure 1: NAMOSIM

Summary

NAMOSIM is a mobile robot motion planner designed for the problem of Navigation Among Movable Obstacles (NAMO). The planner simulates robots navigating in 2D polygonal environments where certain obstacles can be grasped and relocated to enable robots to reach their goals. NAMOSIM extends the classic navigation problem with a layer of interactivity, posing interesting research questions while remaining well-defined and amenable to both classical and learning-based approaches. NAMOSIM supports multi-robot environments and provides a baseline NAMO algorithm along with a communication-free coordination strategy. The simulator includes support for holonomic and differential-drive motion models and integrates with ROS2 for visualization in RViz.

NAMOSIM uses a modular agent-based architecture and includes a baseline NAMO algorithm (Stilman & Kuffner, 2005) implemented in the Stilman2005 agent, which also incorporates a communication-free coordination strategy for multi-robot scenarios. A variety of other agent types are implemented, and new agents utilizing alternative approaches can be created and integrated into the planner by implementing the **Agent** base class. Thus, new navigation algorithms, including those based on machine learning or AI, can be developed within NAMOSIM,

22 facilitating reproducible research on NAMO problems.

23 NAMOSIM utilizes ROS2 messages for visualization of environments and plans using RViz and
24 includes several prebuilt scenarios for testing and benchmarking. Scenarios are stored as SVG
25 files, allowing custom scenarios to be created using a free SVG editor such as Inkscape.

26 NAMOSIM is packaged as a ROS2 package for easy integration into robotics projects but can
27 also be used as a standalone Python module. The package is intended for researchers and
28 developers working on robot navigation in dynamic environments, particularly where physical
29 interaction is necessary.

30 Statement of Need

31 Many applications in autonomous mobile robotics involve physical interaction with the envi-
32 ronment and social coordination with other agents. However, standard navigation planners
33 assume static, non-interactive environments, limiting their applicability in complex real-world
34 scenarios. NAMO problems involve not only path planning but also reasoning about which
35 obstacles to move, where to move them, and how to combine standard navigation with obstacle
36 manipulation. NAMOSIM addresses this gap by offering a simulation environment explicitly
37 designed to study and prototype NAMO algorithms. Additionally, NAMOSIM supports multi-
38 robot environments, facilitating reproducible research in multi-robot navigation among movable
39 obstacles (MR-NAMO).

40 Major Features

41 NAMOSIM provides a robust set of features to support research and development in Navigation
42 Among Movable Obstacles (NAMO):

- 43 ■ **Modular Agent-Based Architecture:** The simulator is built around a flexible Agent
44 interface, allowing users to implement and test custom NAMO planning algorithms. A
45 baseline implementation of the Stilman2005 planner is included for immediate use and
46 benchmarking.
- 47 ■ **Support for Multiple Robot Models:** NAMOSIM supports both holonomic and differential-
48 drive robot models, enabling realistic simulation of various robotic platforms.
- 49 ■ **ROS2 Integration:** Full compatibility with ROS2 allows seamless deployment on both sim-
50 ulated and physical robots, with built-in support for visualization in RViz and integration
51 with ROS2 navigation stacks.
- 52 ■ **2D Environment Simulation:** The simulator provides a customizable 2D environment
53 where users can define static and movable obstacles, supporting complex scenarios for
54 testing navigation and manipulation strategies.
- 55 ■ **Extensive Testing and Documentation Utilities:** NAMOSIM includes tools for automated
56 testing of planning algorithms and generating comprehensive documentation, facilitating
57 reproducible research and educational use.
- 58 ■ **Multi-Robot Coordination:** The simulator supports multi-robot scenarios, enabling the
59 study of implicit coordination and conflict resolution in NAMO tasks, as explored in
60 related research (Renault et al., 2024).

61 These features make NAMOSIM a versatile tool for prototyping, evaluating, and deploying
62 NAMO algorithms in diverse robotic applications.

63 Customizable Scenarios

64 NAMOSIM environments, or **scenarios**, are stored in SVG format and can be edited using any
65 SVG editor, such as Inkscape. The scenario SVG file contains the following key elements:

- 66 ▪ The geometry of the static map
- 67 ▪ The polygons and orientations of all robots and movable obstacles
- 68 ▪ Configuration settings that define the behavior of the environment and robots

69 The static map can also be included as an image layer within the SVG to conveniently
70 incorporate ROS grid-map images generated by standard mapping tools.

71 Architecture

72 At a high level, NAMOSIM executes a SENSE-THINK-ACT loop that performs the following
73 functions at each iteration:

- 74 1. **SENSE**: Each agent senses the environment and updates its internal representation.
- 75 2. **THINK**: Each agent computes a new plan or updates its current plan.
- 76 3. **ACT**: Each agent selects a single discrete action to execute.

77 The loop is expected to execute at a regular frequency, with the assumption that all agent
78 functions run sequentially in a synchronized manner.

79 Stilman's Algorithm

80 NAMOSIM includes a baseline implementation of Stilman's 2005 NAMO algorithm ([Stilman
81 & Kuffner, 2005](#)). The key idea of this algorithm is to move obstacles to merge disjoint
82 components of the robot's free configuration space. The map is divided into a set of disjoint
83 **connected components**, where each cell in a given component is reachable from all other cells
84 in the same component. It can be proven that components are separated by movable obstacles
85 or are otherwise unreachable. The algorithm functions by moving obstacles to join components
86 until the robot's current component includes the goal cell.

87 The algorithm works by recursively performing the following two stages:

- 88 1. **SELECT_OBSTACLE_AND_COMPONENT**: The first stage performs a simplified
89 A* grid search, allowing the agent to pass through movable obstacles. It returns the ID
90 of the first movable obstacle encountered on the optimal path to the goal and the ID of
91 the component encountered after passing through the obstacle.
- 92 2. **OBSTACLE_MANIPULATION_SEARCH**: The second stage finds a **transit path**
93 from the robot's current position to a grasp pose near the obstacle. Then, it finds a
94 **transfer path** by performing an obstacle manipulation search to join the robot's current
95 component to the component selected in stage 1. If this stage fails, the obstacle and
96 component pair are added to an avoid-list, and the algorithm returns to stage 1.

97 Each iteration of the algorithm continues with a copy of the environment where the robot and
98 obstacle start from the poses resulting from the previous obstacle manipulation search. This
99 algorithm is explained in greater detail in Renault's 2023 PhD thesis ([Renault, 2023](#)).

100 Collision Detection

101 Custom agents can implement their own collision detection routines; however, the baseline
102 Stilman2005 agent detects collisions using a simple binary-occupancy grid during transit paths
103 (when not carrying an obstacle), assuming a circular robot footprint. When transporting a
104 movable obstacle, the robot footprint is non-circular, and collision detection is based on the
105 **convex swept volume** resulting from the area swept by the combined robot-obstacle footprint
106 due to the action motion. Although computationally expensive, this ensures all possible
107 collisions are detected, regardless of the shape of the robot or obstacle.

108 Social Costmap

109 A novel contribution in the baseline implementation is the option to use a social costmap
110 during the obstacle manipulation search to guide obstacle placement decisions. This allows
111 robots to place obstacles in areas less likely to block the free passage of other agents, including
112 humans, reducing the likelihood that obstacles will need to be moved again. The key heuristic
113 of the social costmap is to **avoid narrow corridors and central areas**, assigning higher costs to
114 narrow corridors and the centers of open spaces. This helps robots avoid placing obstacles in
115 front of doorways or in the center of rooms. The social costmap is explained in greater detail
116 in Renault (2023).

117 Conflict Avoidance and Deadlock Resolution

118 The baseline Stilman2005 agent can avoid conflicts and resolve deadlocks with other agents.
119 Conflict avoidance works by looking ahead along the agent's current plan for a fixed number
120 of steps, called the **conflict horizon**. Within this horizon, the agent simulates each planned
121 action and checks for potential conflicts, such as an obstacle moved by another robot or a
122 collision with another robot crossing the planned path.

123 The Stilman2005 agent avoids conflicts by either pausing or replanning around them. A
124 deadlock is detected when a conflict configuration is repeatedly encountered, even after
125 replanning. To resolve deadlocks, the agent follows an evasion strategy as described in
126 (Renault et al., 2024).

127 Acknowledgements

128 This research was supported by an Inria ADT initiative. We express our gratitude to Benoit
129 Renault, whose PhD thesis forms the foundation of this work.

130 References

- 131 Renault, B. (2023). *NAvigation en milieu MOdifiable (NAMO) étendue à des contraintes*
132 *sociales et multi-robots* (Theses No. 2023ISAL0105, INSA de Lyon). [https://hal.science/](https://hal.science/tel-04418723)
133 [tel-04418723](https://hal.science/tel-04418723)
- 134 Renault, B., Saraydaryan, J., Brown, D., & Simonin, O. (2024). Multi-robot navigation
135 among movable obstacles: Implicit coordination to deal with conflicts and deadlocks.
136 *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1–7. [https://](https://hal.science/hal-04705395)
137 hal.science/hal-04705395
- 138 Renault, B., Saraydaryan, J., & Simonin, O. (2020). Modeling a social placement cost to
139 extend navigation among movable obstacles (NAMO) algorithms. *IEEE/RSJ International*
140 *Conference on Intelligent Robots and Systems (IROS)*, 11345–11351. [https://doi.org/10.](https://doi.org/10.1109/IROS45743.2020.9340892)
141 [1109/IROS45743.2020.9340892](https://doi.org/10.1109/IROS45743.2020.9340892)
- 142 Stilman, M., & Kuffner, J. J. (2005). Navigation among movable obstacles: Real-time
143 reasoning in complex environments. *International Journal of Humanoid Robotics*, 2(4),
144 479–503. <https://doi.org/10.1142/S0219843605000545>